

ChatGPT - Mid

write me the question and cpp code for job sequence ordering (greedy) in cpp. The job id, deadline and profit should be given in differetn arrays

Sure! Below is a clear **Job Sequencing with Deadline** problem statement and its C++ implementation using greedy strategy.

The input is taken as three separate arrays: **Job IDs**, **Deadlines**, and **Profits**.

Question: Job Sequencing with Deadlines (Greedy) in C++

Given **n jobs**, each with an **ID**, a **deadline**, and a **profit** if the job is finished before its deadline.

Every job takes **single unit of time**.

The goal is to **schedule the jobs to maximize total profit**, ensuring no two jobs overlap in time.

Input:

- `jobID[]` : array of strings or integers (Job IDs)
- `deadline[]` : array of integers (Deadline for each job)
- `profit[]` : array of integers (Profit for each job)

Output:

- Print the sequence of job IDs scheduled for maximum profit and the **total profit**.

Sample Input

cpp

```
jobID[] = {1, 2, 3, 4, 5}
deadline[] = {2, 1, 2, 1, 3}
```

```
profit[] = {100, 19, 27, 25, 15}
```

Expected Output

yaml

Scheduled Jobs: 1 3 5

Total Profit: 142

C++ Code: Job Sequencing with Deadline

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// A structure to represent a job
struct Job {
    int id;
    int deadline;
    int profit;
};

// Comparator function to sort jobs by profit (descending)
bool compare(Job a, Job b) {
    return a.profit > b.profit;
}

void jobSequencing(int jobID[], int deadline[], int profit[], int n) {
    // Step 1: Create an array of jobs
    vector<Job> jobs(n);
    for (int i = 0; i < n; i++) {
        jobs[i].id = jobID[i];
        jobs[i].deadline = deadline[i];
        jobs[i].profit = profit[i];
    }
```

// Step 2: Sort jobs by profit in descending order

```
sort(jobs.begin(), jobs.end(), compare);
```

// Step 3: Find the maximum deadline to determine the number of slots

```
int maxDeadline = 0;
```

```
for (int i = 0; i < n; i++)
```

```
    if (jobs[i].deadline > maxDeadline)
```

```
        maxDeadline = jobs[i].deadline;
```

// Step 4: Create time slots and initialize all slots as free

```
vector<int> slot(maxDeadline + 1, -1); // slot[0] unused
```

```
vector<int> result;
```

```
int totalProfit = 0;
```

// Step 5: Iterate over jobs and schedule each if possible

```
for (int i = 0; i < n; i++) {
```

```
    // Find a free slot for this job (starting from its deadline)
```

```
    for (int j = jobs[i].deadline; j > 0; j--) {
```

```
        if (slot[j] == -1) {
```

```
            slot[j] = jobs[i].id;
```

```
            result.push_back(jobs[i].id);
```

```
            totalProfit += jobs[i].profit;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

// Step 6: Print result

```
cout << "Scheduled Jobs: ";
```

```
for (int id : result)
```

```
    cout << id << " ";
```

```
cout << "\nTotal Profit: " << totalProfit << endl;
```

```
}
```

// Example usage

```
int main() {
```

```
    int jobID[] = {1, 2, 3, 4, 5};
```

```
    int deadline[] = {2, 1, 2, 1, 3};
```

```
    int profit[] = {100, 19, 27, 25, 15};
```

```
    int n = sizeof(jobID) / sizeof(jobID[0]);
```

```
jobSequencing(jobID, deadline, profit, n);  
return 0;  
}
```

You can change the arrays in `main()` to test with other data.

Let me know if you want explanations or further modifications!

give an example walkthrough

Absolutely! Here's a step-by-step walkthrough using the **sample input** from above:

Sample Input

```
jobID[] = {1, 2, 3, 4, 5}  
deadline[] = {2, 1, 2, 1, 3}  
profit[] = {100, 19, 27, 25, 15}
```

- There are **5 jobs**.
- Each job takes **1 unit time**.
- A job **must be completed before or on its deadline** to earn its profit.

Step 1: Create Job List

Make a list of jobs (ID, Deadline, Profit):

Job	Deadline	Profit
1	2	100
2	1	19

Job	Deadline	Profit
3	2	27
4	1	25
5	3	15

Step 2: Sort Jobs by Profit (Descending)

Sorted list:

Job	Deadline	Profit
1	2	100
3	2	27
4	1	25
2	1	19
5	3	15

Step 3: Find Maximum Deadline

- Maximum deadline is 3 (from job 5).
 - So, there are 3 slots: [slot1, slot2, slot3]
-

Step 4: Schedule Jobs (Greedy)

Slot representation:

- slot[1], slot[2], slot[3], all initially free (-1)

1. Job 1 (Deadline=2, Profit=100)

- Check slot 2 (deadline): free.
- Assign Job 1 to slot 2.
- `slot[2] = 1`

2. Job 3 (Deadline=2, Profit=27)

- Check slot 2: already taken by Job 1.
- Check slot 1: free.
- Assign Job 3 to slot 1.
- `slot[1] = 3`

3. Job 4 (Deadline=1, Profit=25)

- Check slot 1: already taken by Job 3.
- No slot available before deadline, skip.

4. Job 2 (Deadline=1, Profit=19)

- Check slot 1: already taken, skip.

5. Job 5 (Deadline=3, Profit=15)

- Check slot 3: free.
- Assign Job 5 to slot 3.
- `slot[3] = 5`

Step 5: Final Schedule

Slot	Job
1	3
2	1
3	5

Scheduled Jobs (in the order assigned): `1 3 5`

(Your output may print them in the order they were picked, e.g., `1 3 5` or `3 1 5` — both are correct as long as jobs fill free slots by max profit.)

Step 6: Total Profit Calculation

Add up profits for the scheduled jobs:

- Job 1: 100
- Job 3: 27
- Job 5: 15

Total Profit = 100 + 27 + 15 = 142

Output

yaml

Scheduled Jobs: 1 3 5

Total Profit: 142

Summary:

- Always try to put the highest profit job in its latest possible free slot (before or on deadline).
- Greedy choice = highest profit first.
- Skip jobs if no slot is available.

Let me know if you want to try with a different example or need the step-by-step C++ code output with debug!

give pseudocode

Certainly! Here's clear and **well-commented pseudocode** for the **Job Sequencing with Deadline** (Greedy) problem, using separate arrays for Job IDs, Deadlines, and Profits.

Job Sequencing with Deadlines – Pseudocode

java

```
function JobSequencing(jobID[], deadline[], profit[], n):

    // Step 1: Create a list of jobs as (id, deadline, profit) tuples
    jobs = []
    for i = 0 to n-1:
        jobs.append( (jobID[i], deadline[i], profit[i]) )

    // Step 2: Sort the jobs by profit in descending order
    sort jobs by profit in decreasing order

    // Step 3: Find the maximum deadline to determine the number of slots
    maxDeadline = maximum value in deadline[]

    // Step 4: Create an array to keep track of free slots
    slot = array of size (maxDeadline + 1), initialized with False (means free)
    scheduledJobs = empty list
    totalProfit = 0

    // Step 5: Iterate through all jobs
    for each job in jobs:
        // Try to find a free slot from job.deadline down to 1
        for j = job.deadline downto 1:
            if slot[j] is False:
                slot[j] = True // Mark slot as occupied
                scheduledJobs.append(job.id)
                totalProfit = totalProfit + job.profit
                break // move to next job

    // Step 6: Output scheduled jobs and total profit
    print "Scheduled Jobs:", scheduledJobs
    print "Total Profit:", totalProfit
```

How it works

1. **Represent jobs** with IDs, deadlines, and profits.
 2. **Sort** by profit (greedy).
 3. **Try to schedule** each job in the latest available slot before its deadline.
 4. **Collect scheduled jobs** and sum up their profits.
-

Let me know if you need this in flowchart, image, or stepwise explanation for interviews!